

Compose Navigation Architecture Guide

Model: google/gemma-4-e4b Created: 4/27/2026, 6:11:36 PM

User

Расскажи Про навигацию в kotlin compose

Assistant

Конечно! Тема навигации в Jetpack Compose — это одна из самых важных и часто вызывающих вопросы тем, потому что она требует смены мышления: мы переходим от императивного подхода (Activity A вызывает Activity B) к **декларативному** подходу (UI зависит от текущего состояния).

Если говорить о "Про" навигации, то речь идет не просто о вызове `navController.navigate("route")`, а об архитектурном подходе к управлению состоянием приложения через навигационный граф и соблюдении лучших практик MVVM/MVI.

Вот подробный обзор ключевых концепций, инструментов и продвинутых паттернов.

Фундаментальные инструменты (The Basics)

Все начинается с библиотеки `androidx.navigation:navigation-compose`.

1. NavController

Это главный контроллер навигации. Он отвечает за отслеживание текущего состояния, управление стеком экранов и выполнение самих переходов.

```
// Получаем экземпляр контроллера в области видимости Composable
val navController = rememberNavController()
```

2. NavHost

Это компонент-хозяин, который знает о всех возможных маршрутах (Routes) и отображает соответствующий Composable на основе текущего состояния navController.

```
@Composable
fun AppNavHost(navController: NavHostController) {
    NavHost(
        navCount = 1, // Тут нужно указать начальный экран
        startDestination = "home",
        navController = navController
    ) { backStackEntry ->
        composable("home") { HomeScreen(onNavigateToDetails = { ...
    }) }

        composable("details/{itemId}") { backStackEntry ->
            // Обработка аргументов: получение ID из пути
            val itemId =
                backStackEntry.arguments?.getString("itemId") ?: ""
            DetailScreen(itemId = itemId)
        }
    }
}
```

✨ Продвинутые концепции (The "Pro" Level)

На уровне эксперта вы должны думать не о *переходах*, а о *состоянии* приложения.

1. Управление аргументами (Typed Arguments)

Вместо того чтобы передавать ID как строку и парсить его на каждом экране, используйте структурированные аргументы. Compose Navigation позволяет работать с типами данных в пути:

```
// 1. Определение маршрута с типом параметра
composable(
    route = "product/{productId}/{category}",
    arguments = listOf(navArgument("productId") { type =
NavType.StringType })
) { backStackEntry ->
    // 2. Безопасное получение аргумента нужного типа
    val productId: String =
        backStackEntry.arguments?.getString("productId") ?: ""
```

```
        ProductScreen(productId = productId)
    }

    // Навигация:
    navController.navigate("product/$id/electronics")
```

2. Использование Router (Центральный диспетчер)

Самая частая ошибка новичков — это когда каждый экран знает, как ему перейти на следующий экран. **Правильная архитектура:** создать отдельный класс-диспетчер (Router), который будет отвечать за все переходы и знать о всех возможных маршрутах.

Пример: Вместо того чтобы передавать колбэк `onNavigateToDetails` в `HomeScreen`:

```
// Плохо: HomeScreen сам вызывает навигацию
@Composable
fun HomeScreen(navController: NavController) {
    Button(onClick = { navController.navigate("details/$id") }) {}
}
```

Используйте Router (паттерн Mediator/Coordinator):

Создайте `AppRouter` или передавайте не `NavController`, а *функцию-обработчик* из родительского компонента, который знает об общем контексте приложения. Это делает экраны максимально независимыми от механизма навигации.

3. Управление стеком (Back Stack Management)

Иногда нужно, чтобы при возврате назад пользователь попадал не только на предыдущий экран, но и сохранялась часть данных или выполнялись какие-то действия (например, обновление списка).

- **popUpTo:** Позволяет "выгрузить" несколько экранов из стека.

```
// Переходим на профиль, полностью удаляя Home Screen со стека
navController.navigate("profile") {
    popUpTo("home") { inclusive = true } // Удаляем всё до
    'home'
}
```

- **launchSingleTop:** Гарантирует, что если вы пытаетесь перейти на экран, который уже находится в стеке, вы не создадите новую копию этого экрана.

4. Deep Linking (Глубокие ссылки)

Это критически важно для интеграции с внешними системами (браузеры, push-уведомления). Compose Navigation позволяет привязать URL к определенному маршруту:

1. **В `nav_graph.xml` или коде:** Объявляете схему ссылки.
2. **Обработка в `NavHost`:** Навигация автоматически обрабатывает входящую ссылку и переходит по нужному экрану, как если бы пользователь нажал кнопку внутри приложения.

5. Состояние при возврате (State Restoration)

Когда вы покидаете экран (Screen A) и затем возвращаетесь к нему из Screen B, данные могут быть потеряны.

Решение:

1. **Сохранение состояния в `ViewModel`:** Перемещайте все критически важные данные в `ViewModel` привязанный к жизненному циклу этого `Composable`. Даже если Compose уничтожит и восстановит `Composable`, `ViewModel` останется живым.
2. **Использование `StateFlow/LiveData`:** Используйте реактивные потоки для передачи данных от бизнес-логики (`ViewModel`) прямо в UI, не полагаясь на параметры навигации.

Сводная таблица: "Про" vs. Новичок

Аспект	Подход новичка (Beginner)	Профессиональный подход (Pro/Architectural)
Передача данных	Паровые аргументы в пути (<code>details/{id}</code>), примитивы.	Использование <code>ViewModel</code> как источника истины. Аргументы — только для идентификаторов.
Управление переходами	Каждый <code>Composable</code> вызывает навигацию сам (спагетти-код).	Центральный <code>Router/Coordinator</code> , который инкапсулирует всю логику переходов и знает все маршруты.
Состояние экрана	Полагается только на параметры <code>backStackEntry</code> .	Использует <code>ViewModel</code> для сохранения состояния независимо от жизненного цикла <code>Composable</code> .
Обработка ошибок	Нет обработки или простейшие <code>try/catch</code> .	Внедрение централизованной системы обработки ошибок (e.g., <code>Result wrapper</code>) и обработчиков путей.

Аспект	Подход новичка (Beginner)	Профессиональный подход (Pro/Architectural)
Тестирование	Сложно протестировать навигацию.	Навигационный граф и Router легко поддаются юнит-тестам, так как логика переходов вынесена в класс.